ETHEREUM PEERDAS · CODING SCHEME COMPARISON

Benchmarking 1D-RS, 2D-RS, and BC codes

How the code works, what it costs, and what it tolerates

Benchmarking 1D-RS, 2D-RS, and Block-Circulant codes

A controlled comparison of three erasure-coding schemes as candidates for Ethereum's data-availability sampling: one-dimensional Reed-Solomon (what PeerDAS uses today), two-dimensional Reed-Solomon, and Block-Circulant (BC) codes. All three run over the same field, on the same blob, under the same erasure model, and are judged by the same success criterion.

The question each scheme has to answer is the same: **if a fraction of the encoded data goes missing, can the original blob still be reconstructed exactly — and what did that cost in storage and in decode time?**

How the code works

The whole experiment is five moving parts. Each is a separate file, and the benchmark drives them without reimplementing any of their logic.

Packing a blob (`blob_generator.py`). An Ethereum blob is 4096 field elements of the BLS12-381 scalar field. Each element is 32 bytes on the wire but only 31 are usable, because the value must stay below the field modulus — so one blob carries 126,976 bytes of application data. This is the unit everything else operates on: a list of 4096 field elements, and a function to turn it back into the original bytes.

1D-RS (`1D_RS_Galois.py`). The key idea is that the blob's 4096 elements are not raw data to be copied — they are the *values of a single polynomial* sampled at 4096 fixed points. Encoding recovers that polynomial (an inverse NTT turns 4096 values into 4096 coefficients), pads it with zeros, and evaluates it again at 8192 points (a forward NTT). The result is 8192 symbols describing the same polynomial. Since a polynomial of degree below 4096 is pinned down by *any* 4096 of its values, any 4096 surviving symbols reconstruct everything. Decoding builds an erasure-locator polynomial whose roots sit exactly at the missing positions, multiplies through by

it, inverts the transform, and divides the locator back out — recovering the original polynomial in one pass. Crucially it first checks whether the number of erasures exceeds `n−k`; if so it gives up immediately rather than attempting anything.

2D-RS (`2D_RS_Galois.py`). The same interpolate-and-extend step, applied twice. The blob is reshaped into a 64×64 grid; every row is extended to 128, then every column of that result is extended to 128, giving a 128×128 codeword at 4× overhead. Because the original grid is a single bivariate polynomial, every row *and* every column of the result is independently a valid 1D-RS codeword. Decoding exploits this: it sweeps all columns, then all rows, repeatedly. A line holding too many erasures to fix alone often becomes fixable once the perpendicular sweep has repaired some of its cells, so repairs cascade across the two axes until nothing more can be done.

Block-Circulant (`BCcodeEthereum.py`). Structurally different: instead of one global polynomial, the message is cut into $\mu=2048$ small blocks, and each block gets $\rho=2$ parity symbols computed from its own information plus that of $\lambda=2$ consecutive neighbours, wrapped around a ring. Overhead is 2×, the same as 1D-RS, but every parity symbol is *local*. Decoding is correspondingly local: a block missing at most ρ symbols is solved from its own parity equations, and an adjacent pair missing at most 2ρ between them can sometimes be solved jointly. Both are tiny fixed-size linear solves, which is why BC decodes far faster than either RS variant — but also why its tolerance depends on erasures staying thinly spread.

The harness (`benchmark_1D_2D_BC.py`). Each scheme is wrapped in a small adapter exposing the same five operations — `encode`, `random_mask`, `corrupt`, `decode`, `extract_message` — so the sweep loop never needs to know that 2D-RS's codeword is a grid while BC's is a flat vector. The adapters call the real functions in the files above; no coding logic is duplicated, so fixing one of those files changes the benchmark's results automatically.

The experimental pipeline

Every measurement follows the same seven steps, and only two of them differ between schemes:

1. **Generate** 126,976 random application bytes.
2. **Pack** them into one blob of 4096 field elements, the real EIP-4844 way.
3. **Encode** — *scheme-specific* — expanding 4096 elements into a codeword of `n` elements.
4. **Erase** — withhold each codeword position independently with probability equal to the target rate, simulating samples that peers failed to provide.
5. **Decode** — *scheme-specific* — attempt to reconstruct the full codeword from what survived.
6. **Extract** the original 4096 elements back out of the recovered codeword.
7. **Verify** by unpacking to bytes and comparing against step 1.

Steps 1, 2, 4, 6 and 7 are byte-identical across all three schemes. That is what makes the comparison fair: any difference in the results has to come from step 3 or step 5, which is precisely what is being compared.

Step 7 is the one worth dwelling on. Success is defined as *the recovered bytes equal the original bytes* — not as "the decoder reported success", and not even as "the field elements matched". That distinction is not pedantry: one of the three decoders turned out to report success on data it had silently corrupted, and only the byte comparison caught it.

What is held identical

- **Field** — the real BLS12-381 scalar field.
- **Message** — the same 4096 elements, packed from the same bytes, fed to all three encoders. Not three different random blobs.
- **Erasure model** — independent per-position withholding at a shared target rate. 1D-RS and BC share `n = 8192`, so on each trial they are corrupted at the *same literal positions*, not merely the same rate.
- **Success criterion** — the full byte-level round trip described above.

What is deliberately *not* equalised is each scheme's own shape: 1D-RS and BC store 2× the blob, 2D-RS stores 4×. That difference is one of the things being compared, so it is reported rather than controlled away — and it is the reason the two figures below have to be read together.

Method

Erasure rates are swept from 5% to 60%. Trial counts vary by scheme and rate: a scheme decoding against a formal budget check behaves deterministically well away from its threshold, so one trial there is representative, while near a threshold the outcome genuinely varies from draw to draw and needs several. Trials are therefore concentrated around each scheme's own transition — which is not the same rate for all three. Each success rate carries a 95% Wilson confidence interval.

Code parameters and storage cost

All three carry the same 124 KiB blob payload; they differ in how much they store to protect it. `R = k/n` is the code rate, and `1−R` is the largest erasure fraction *any* code of that rate could survive.

	1D-RS	2D-RS	BC
Message k (field elements)	4096	4096	4096
Codeword n (field elements)	8192	16384	8192
Code rate R = k/n	0.50	0.25	0.50
Storage overhead	2×	4×	2×
Blob payload	124 KiB	124 KiB	124 KiB
Stored per blob	256 KiB	512 KiB	256 KiB
Parity added	128 KiB	384 KiB	128 KiB
Theoretical erasure limit (1−R)	0.50	0.75	0.50

Encode and decode cost, from the same run. Decode times are the mean over trials that actually recovered the blob; a dash means nothing recovered at that rate, so there is no decode time to report.

	1D-RS	2D-RS	BC
Encode	0.47s	3.15s	0.94s
One-time setup	—	—	0.94s
Decode @ 5% erasure	11.72s	4.16s	0.15s
Decode @ 15% erasure	31.09s	4.98s	0.41s
Decode @ 25% erasure	47.66s	5.71s	—
Decode @ 45% erasure	78.96s	10.51s	—
Decode @ 58% erasure	—	10.85s	—

Results

Recovered trials at each erasure rate. Trial counts differ by scheme and by rate — more where a scheme is near its own threshold and the outcome genuinely varies, fewer where it does not — so each cell shows recovered/attempted rather than a bare percentage.

ERASURE RATE	1D-RS	2D-RS	BC
5%	1/1	3/3	10/10
8%	1/1	3/3	9/10
10%	1/1	3/3	10/10
12%	1/1	3/3	6/10
15%	1/1	3/3	6/10
25%	1/1	3/3	0/10
35%	1/1	3/3	0/10
45%	2/2	5/5	0/10
48%	3/3	5/5	0/10
50%	1/3	5/5	0/10
52%	0/3	5/5	0/10
55%	0/2	5/5	0/10
58%	0/2	5/5	0/10
60%	0/2	1/5	0/10

Across the whole sweep, no scheme returned corrupted data while reporting success: silent corruption was 0/24 for 1D-RS, 0/56 for 2D-RS, 0/140 for BC. Before the decoder fix described below, BC's figure was 31 of 40.

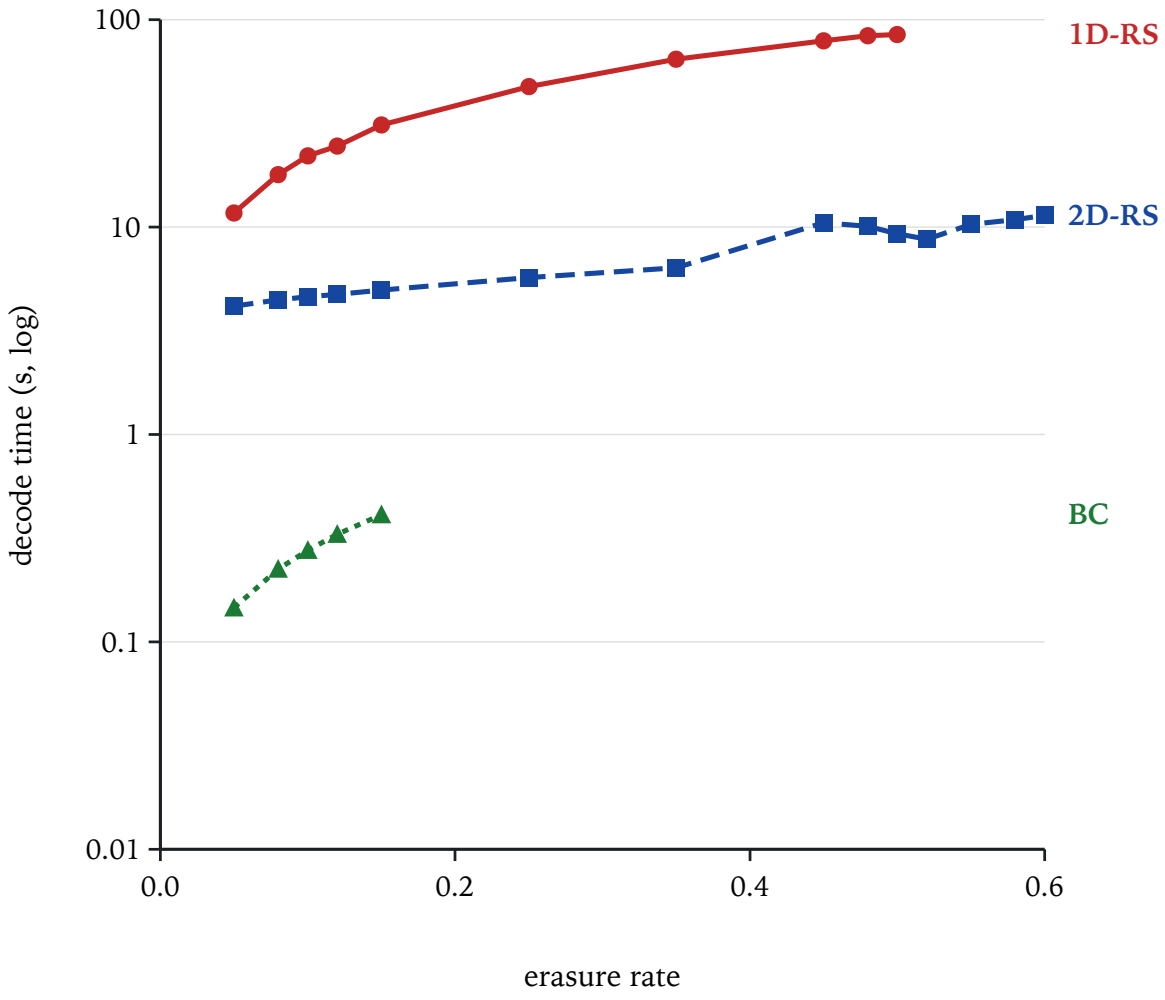


Fig. 1. Decode time against erasure rate, log scale. Only trials that actually decoded are plotted: above its threshold 1D-RS rejects the erasure pattern on a budget check and returns in under a millisecond having attempted nothing, which is not a decode time. 1D-RS pays the most because it divides one 8192-length polynomial; 2D-RS divides 128 short ones instead; BC only ever solves tiny fixed-size systems.

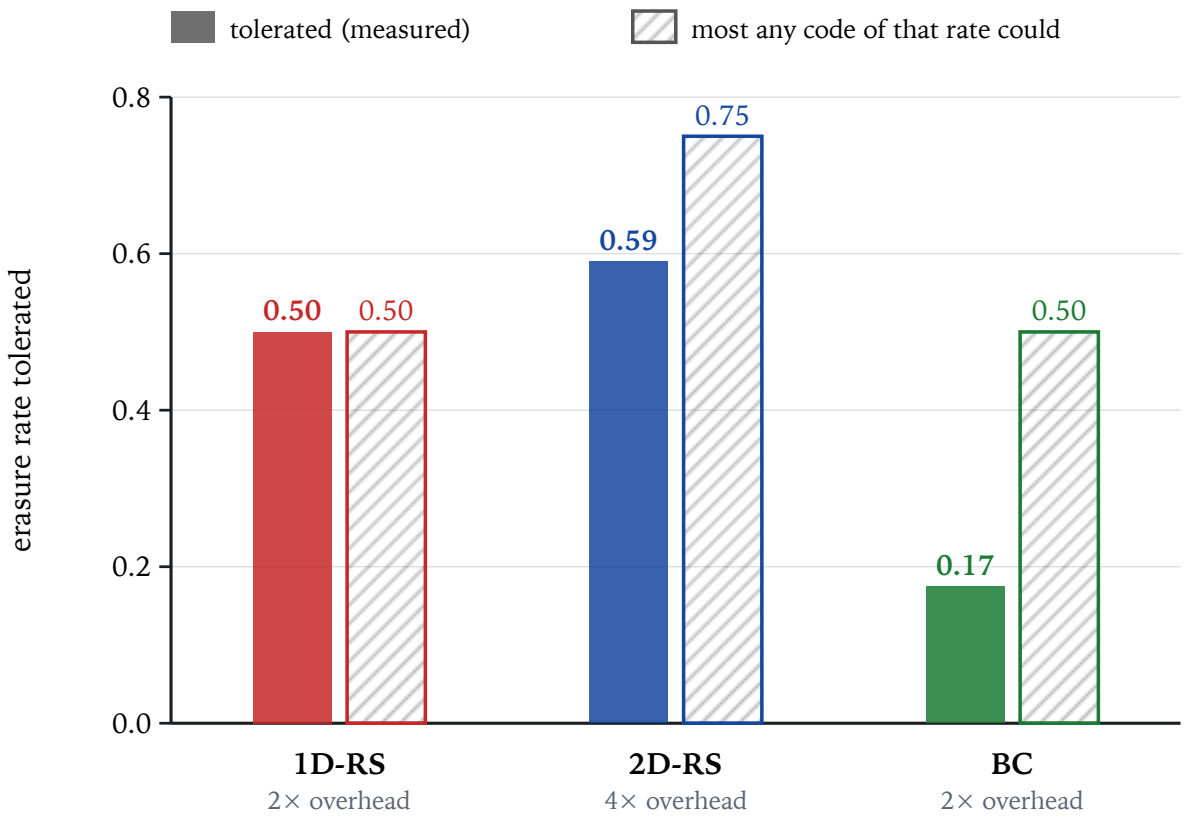


Fig. 2. What each scheme actually tolerated (solid) beside the most any code of the same rate could tolerate (hatched). The two differ because the schemes do not spend the same storage: at 2× overhead the ceiling is 0.50, at 4× it is 0.75. 1D-RS reaches its ceiling exactly — that is what being MDS means. 2D-RS tolerates a higher absolute rate than 1D-RS but falls short of its own, higher ceiling, so the extra tolerance is bought with storage rather than with a better code. BC falls far short of a ceiling it shares with 1D-RS. Solid bars are the midpoint of the measured transition from full recovery to none.

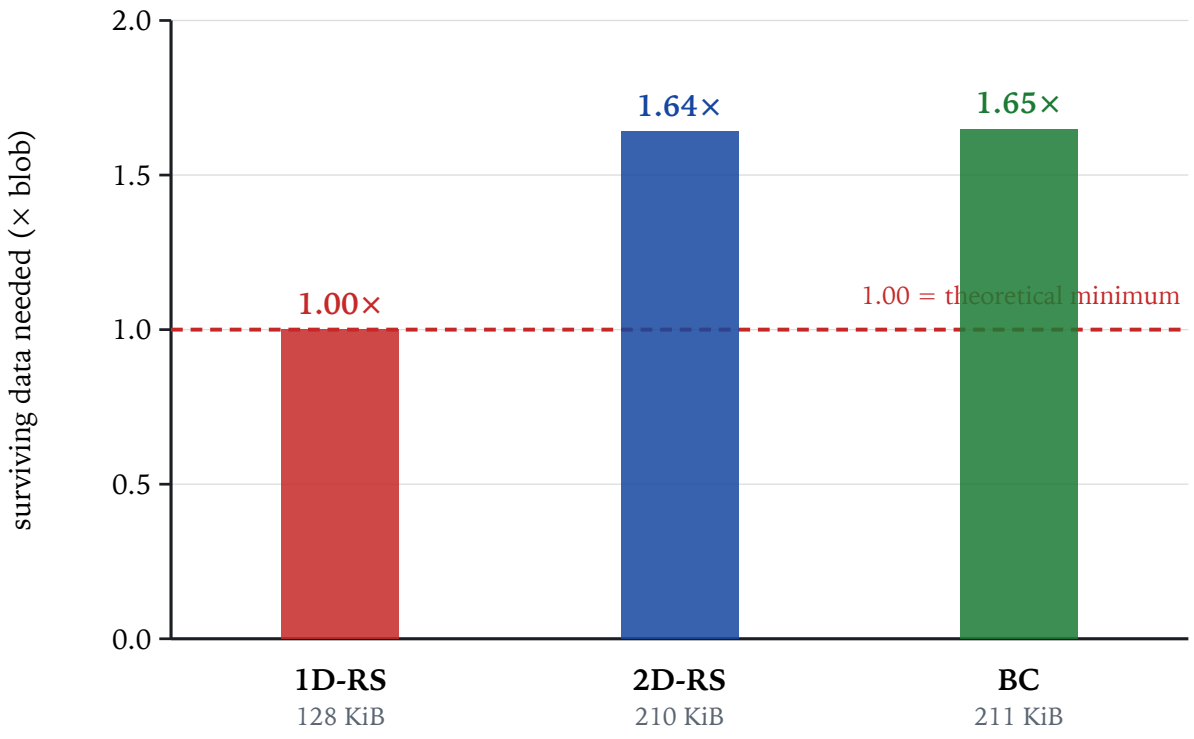


Fig. 3. How much surviving data each scheme needs to rebuild one blob, as a multiple of the blob itself. This is a different question from Fig. 2 and it reorders the schemes. 1D-RS needs exactly one blob's worth of surviving symbols — the theoretical floor, since no code can rebuild 124 KiB from less than 124 KiB. 2D-RS tolerates the highest erasure *rate* of the three, but because it stores 4× it still needs 1.64× the blob in survivors: its rate advantage does not translate into needing less data. BC needs essentially the same (1.65×) despite storing half as much. For data-availability sampling, where a node must actually fetch those symbols, this is the bandwidth-relevant number.

What the results show

1D-RS behaves exactly as Reed-Solomon theory says it must. It recovers everything up to 48% and nothing from 52% upward. The invariant is exact rather than approximate: across every 1D-RS trial in the sweep, recovery succeeded if and only if the drawn erasure count was at most $n-k = 4096$, with no exceptions. The partial results at the threshold are not the decoder being unreliable — they are the random mask landing on either side of a hard line. Being MDS, 1D-RS extracts essentially all of its theoretical budget.

2D-RS survives to a higher rate, but buys that with storage. It holds up to 58% and fails from 60%, well past 1D-RS. That is a real effect: because every row and column is its own codeword, lines that are individually over budget get rescued once their partner axis repairs enough neighbours, and the repair cascades. But 2D-RS runs at 4× overhead, so its own limit is 0.75,

not 0.50. Measured against that, it uses its budget *less* efficiently than 1D-RS uses its own. Comparing the two at equal erasure rate flatters 2D-RS; the second figure is the like-for-like view.

BC is the weakest of the three at this parameterisation. It recovers reliably to 10% and fails from 25%. That is not surprising for its parameters: with $\omega=2$, $\rho=2$, $\lambda=2$ the code's guaranteed correction is only $d-1 = 4$ erasures across the whole 8192-symbol codeword. Everything beyond that depends on erasures staying thinly enough spread that each local block keeps at most ρ of them, which random erasure stops providing well before the rates the RS variants reach. Its decode is by far the cheapest, because it never touches a large polynomial — every step is a 2×2 or 4×4 solve regardless of blob size.

The Block-Circulant decoder fix

The BC results above are from a corrected decoder. The version originally benchmarked had two faults that together made its output untrustworthy rather than merely weak.

`gf_solve` fabricated values. The Gaussian elimination routine returned zero for any unknown whose column never received a pivot. To the caller that is indistinguishable from having genuinely solved for zero, so an underdetermined system silently produced a confident-looking wrong answer. It now raises `GFSolveError` unless the system has exactly one solution — every column pivoted, and no row reduced to `0 = nonzero`.

`dec_cbc_2`'s pairwise phase accepted systems it had not solved. Its guard is `0 < erasures ≤ 2ρ`, which is necessary but not sufficient. The four equations it stacks are block *i*'s own ρ parity checks on top of block *i+1*'s — not the auxiliary `[2(ω+ρ), 2ω, 2ρ+1]` GRS code over the union of the two supports that the source paper's Algorithm 1 constructs. The stacked form is not MDS, so a split such as "three erasures inside block *i*'s support, one inside *i+1*'s" leaves two equations facing three unknowns. Combined with the `gf_solve` fault, the decoder wrote zeros into those positions *and cleared their erasure flags*, reporting success on corrupted data. It now skips any pair it cannot uniquely solve, leaving those positions honestly flagged.

The practical effect is stark. Measured before the fix, across 40 trials spanning 5–45% erasure, `dec_cbc_2` claimed success **every single time** and was wrong on 31 of them; it never once reported a failure. Its `remaining` flag — the signal a caller would naturally trust — carried no information about whether the data was correct. Had this benchmark trusted that flag instead of checking bytes, it would have reported BC at 100% success at every rate.

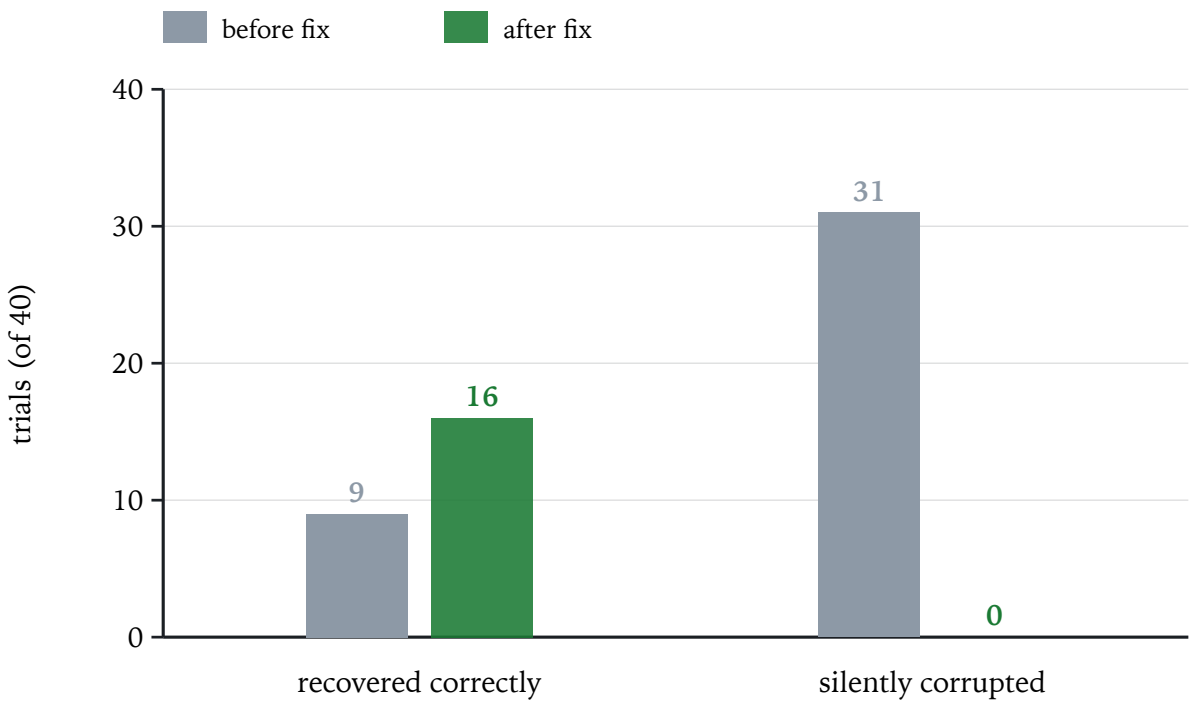


Fig. 4. The same 40 trials (5%, 15%, 25% and 45% erasure, ten each) run through the decoder before and after the fix. Correct recoveries rose from 9 to 16, because the old decoder poisoned positions with fabricated zeros that blocked later rounds from resolving them. Silent corruptions — cases where the decoder reported success on wrong data — fell from 31 to zero. The remaining failures are honest: the decoder reports the positions it could not resolve.

A third change is a performance fix rather than a correctness one: the pairwise phase used to `break` and restart the single-block phase after every pair it repaired, paying a full scan of all 2048 blocks per repair. It now continues through the remaining pairs in the same pass, which cut the iteration count from several hundred to under ten at the rates where BC still succeeds.

What the fix does not do is restore the correction radius the paper claims. Implementing the auxiliary GRS construction would let the pairwise phase handle *any* $\leq 2\rho$ erasures across two adjacent blocks, rather than only those that happen to be solvable from the stacked parity checks. BC's measured tolerance here should therefore be read as a property of this implementation's radius-2 strategy, not of block-circulant codes in general — the literature expects them to beat 2D-RS at matched overhead, which is the opposite of what this implementation does.

Reproducing

```
pip install galois numpy
python3 benchmark_1D_2D_BC.py
```

Roughly 20–30 minutes, dominated by 1D-RS's decode at the higher rates. Per-trial results — erasure counts, decode times, success flags, and the decoder's own claim separately from ground truth — are written to `benchmark_results.json`.

Generated from the project repository's `docs/` source Markdown.